

LAB EXPERIMENT REPORT

Object-Oriented Programming using Java

Basic Information	
Experiment No.	_____
Date	_____
Title	Thread-Safe Seat Reservation System using Synchronized Methods in Java

1. Objective / Aim

To develop a multithreaded Java program in which several users try to reserve seats at the same time, and to use the synchronized keyword so that only one thread updates the shared seat count at a time, preventing race conditions and overbooking.

2. System Requirements

Hardware Requirements

- Processor: Intel Core i3 / AMD equivalent or higher (multi-core preferred)
- RAM: 4 GB minimum (8 GB recommended)
- Hard disk: 500 MB free space
- Input/Output: Keyboard, monitor

Software Requirements

- Operating System: Windows 10/11, Linux or macOS
- Java Development Kit (JDK) 8 or higher
- Editor / IDE: Eclipse, IntelliJ IDEA, NetBeans or VS Code
- Command Prompt / Terminal for compilation and execution

3. Algorithm / Logic Description

Key idea: A single `SeatReservation` object (5 seats) is shared by three User threads. If two threads read and update `availableSeats` simultaneously, both may see the same value and book more seats than exist (a race condition). Declaring `reserveSeat()` as synchronized makes each thread acquire the object's lock before entering, so the check-and-update happens as one indivisible step.

Algorithm:

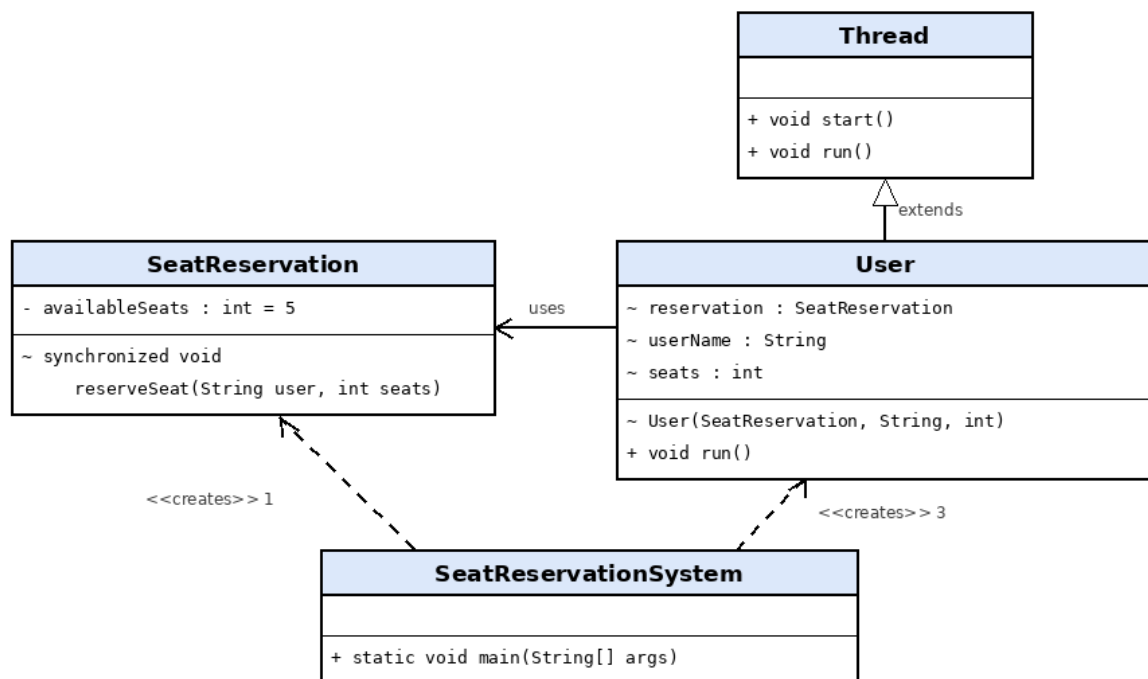
1. Start.
2. Create one shared `SeatReservation` object with `availableSeats = 5`.
3. Create three User threads (Rahul, Priya, Arjun), each requesting 2 seats, sharing that object.
4. Start all three threads.

5. Each thread calls reserveSeat(user, seats). Only one thread can run inside this synchronized method at a time; the others wait for the lock.
6. Inside the method: if seats <= availableSeats, then subtract seats from availableSeats and print the success message and remaining seats.
7. Otherwise, print that seats are not available and show the current availableSeats.
8. The thread releases the lock; the next waiting thread enters.
9. When all threads finish, the program ends. Stop.

Expected behaviour: Total demand is 6 seats but only 5 exist, so exactly two users succeed (5 -> 3 -> 1) and the third is refused. Which user is refused can change from run to run because thread scheduling is not fixed; the seat count never goes negative.

Concepts used: Thread class (extends Thread), start() and run(), synchronized methods, object-level locking, shared resource.

4. UML Class Diagram



Legend: + public - private ~ package-private

5. Source Code

```
class SeatReservation {

    private int availableSeats = 5;

    synchronized void reserveSeat(String user, int seats) {

        System.out.println(user + " is trying to reserve "
            + seats + " seat(s).");

        if (seats <= availableSeats) {

            System.out.println("Seats available: " + availableSeats);

            availableSeats = availableSeats - seats;

            System.out.println(user + " successfully reserved "
                + seats + " seat(s).");

            System.out.println("Remaining seats: "
                + availableSeats);

        } else {

            System.out.println("Sorry " + user
                + ", seats are not available.");

            System.out.println("Available seats: "
                + availableSeats);

        }

        System.out.println();

    }

}

class User extends Thread {

    SeatReservation reservation;
    String userName;
    int seats;

    User(SeatReservation reservation, String userName, int seats) {
        this.reservation = reservation;
        this.userName = userName;
        this.seats = seats;
    }

    public void run() {
        reservation.reserveSeat(userName, seats);
    }

}

public class SeatReservationSystem {

    public static void main(String[] args) {

        SeatReservation reservation = new SeatReservation();

        User user1 = new User(reservation, "Rahul", 2);
        User user2 = new User(reservation, "Priya", 2);
        User user3 = new User(reservation, "Arjun", 2);

        user1.start();
        user2.start();
        user3.start();

    }

}
```

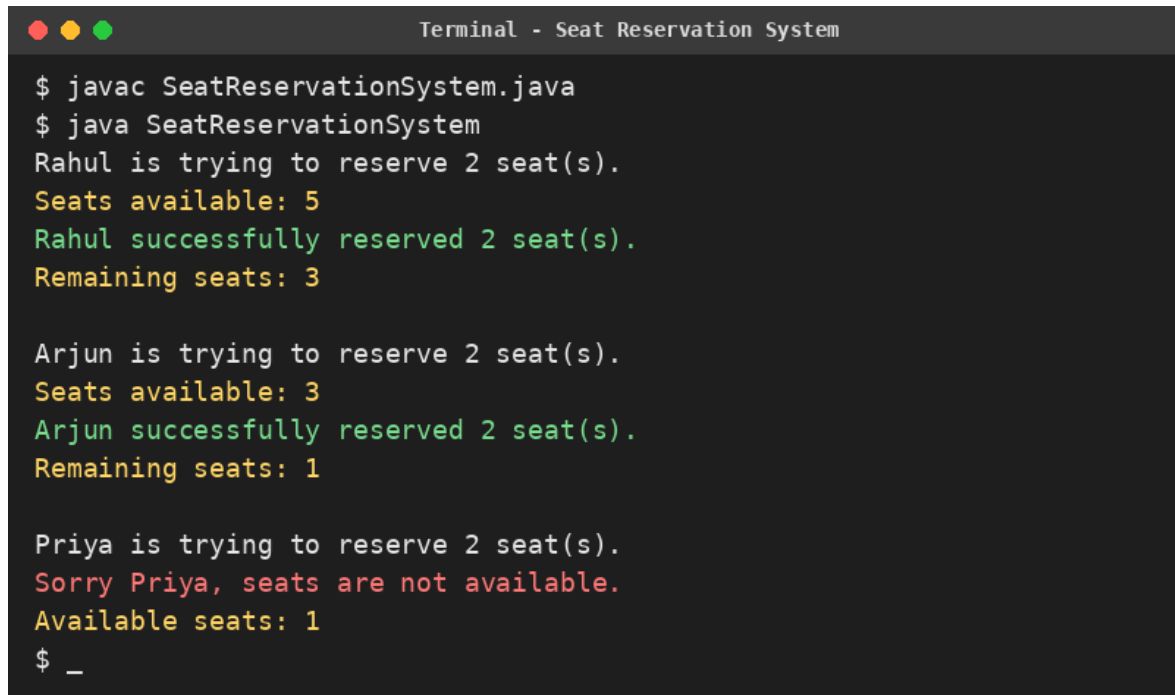
6. Compilation and Execution Commands

Save the program as **SeatReservationSystem.java** (the file name must match the public class).

Open a terminal in that folder and run:

```
javac SeatReservationSystem.java
java SeatReservationSystem
```

7. Output



```
Terminal - Seat Reservation System

$ javac SeatReservationSystem.java
$ java SeatReservationSystem
Rahul is trying to reserve 2 seat(s).
Seats available: 5
Rahul successfully reserved 2 seat(s).
Remaining seats: 3

Arjun is trying to reserve 2 seat(s).
Seats available: 3
Arjun successfully reserved 2 seat(s).
Remaining seats: 1

Priya is trying to reserve 2 seat(s).
Sorry Priya, seats are not available.
Available seats: 1
$ _
```

Observation: Rahul and Arjun reserved 2 seats each (5 -> 3 -> 1). Priya's request failed because only 1 seat remained. The order of users may differ on another run, but the seat count is always consistent.

8. Result / Conclusion

The program shows that a synchronized method allows multiple threads to safely share one resource. Seats were never overbooked, and the available count stayed correct even though three threads ran concurrently.